



Proyecto #1

Lenguajes de Programación

Profesor Bryan Tomás Hernández Sibaja

Preparado por:

Feng Liu Jimmy

Kuo Liu Francisco

Wu Zhong Danielo

Tecnológico de Costa Rica

09/04/2025

Tabla de Contenidos

Enlace de GitHub.....	3
Pasos de Instalación del Programa.....	3
Manual de Usuario.....	5
Funcionalidad del Servidor.....	8
Estructuras y variables globales del Servidor:.....	8
Lógica del servidor y conexiones(Uso de forks).....	10
Flujo Principal del Servidor.....	10
Memoria Compartida.....	12
Lógica de registro y login de usuarios.....	14
Lógica de recepción y envío de mensajes.....	14
Lógica de consulta de información de un usuario por correo.....	15
Funcionalidad del Cliente.....	15
Estructuras y Variables Globales del Cliente.....	15
Enviar y Recibir Mensajes.....	17
Cargar y Guardar Contactos.....	18
Carga y Guardado de Chats.....	19
Interacción con el Cliente.....	21
Comunicación entre Cliente-Servidor.....	22

Enlace de GitHub

[Proyecto 1 de Lenguajes de Programación](#)

Pasos de Instalación del Programa

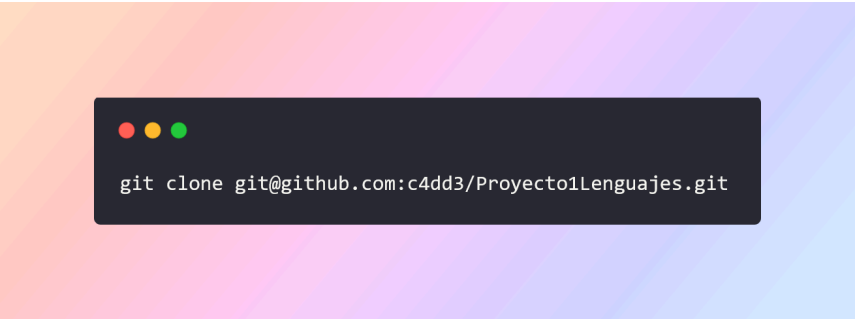
Antes de instalar y ejecutar el programa, asegúrate de tener instalados los siguientes requisitos:

- Git: Necesario para clonar el repositorio desde GitHub.
- Compilador de C++: Para compilar el programa (ej. g++ en Linux o Visual Studio en Windows).
- GTK: Biblioteca gráfica para la interfaz de usuario.

Instrucciones de instalación:

1. Clonar el repositorio desde GitHub

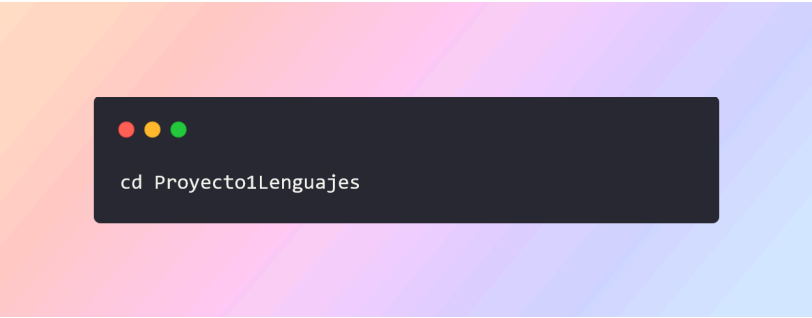
Abre una terminal en linux y clona el repositorio con el siguiente comando:

A screenshot of a terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays the command `git clone git@github.com:c4dd3/Proyecto1Lenguajes.git` in a light-colored monospace font.

```
git clone git@github.com:c4dd3/Proyecto1Lenguajes.git
```

2. Acceder al directorio del proyecto

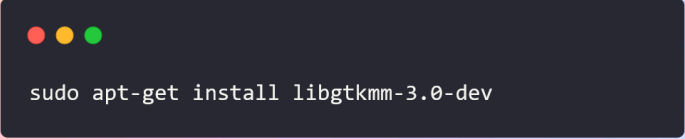
Una vez clonado el repositorio, navega al directorio del proyecto:

A screenshot of a terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal displays the command `cd Proyecto1Lenguajes` in a light-colored monospace font.

```
cd Proyecto1Lenguajes
```

3. Instalar dependencias

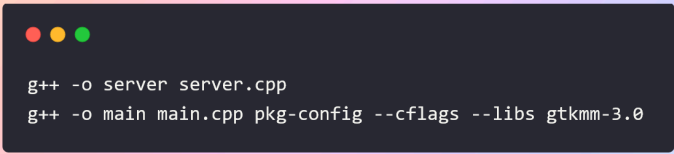
Asegúrate de tener las bibliotecas necesarias instaladas en tu sistema. Si usas un sistema basado en Debian (como Ubuntu), puedes instalar GTK con:



```
sudo apt-get install libgtkmm-3.0-dev
```

4. Compilar el programa

Compila el programa utilizando g++ o tu compilador preferido. Asegúrate de incluir las rutas necesarias para las dependencias de GTK



```
g++ -o server server.cpp  
g++ -o main main.cpp `pkg-config --cflags --libs gtkmm-3.0`
```

5. Ejecutar el programa

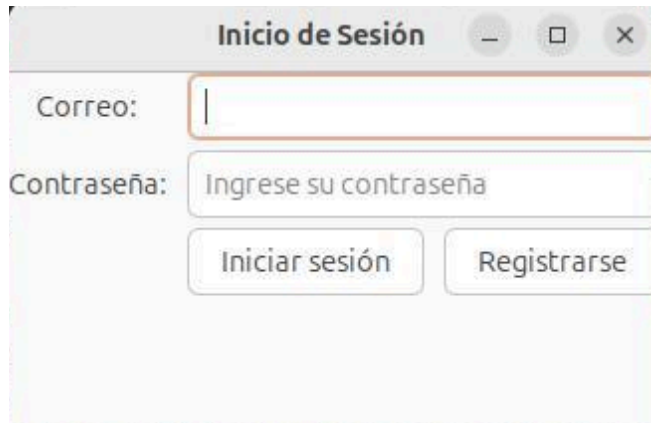
Una vez compilado, se ejecuta primero el servidor y luego el main:



```
./server  
./main
```

Manual de Usuario

Esta es la pantalla de iniciar sesión donde el usuario ingresa su correo y contraseña, al ingresar las credenciales correctas y darle click en el botón de iniciar sesión, lleva al usuario a la pantalla de chats.



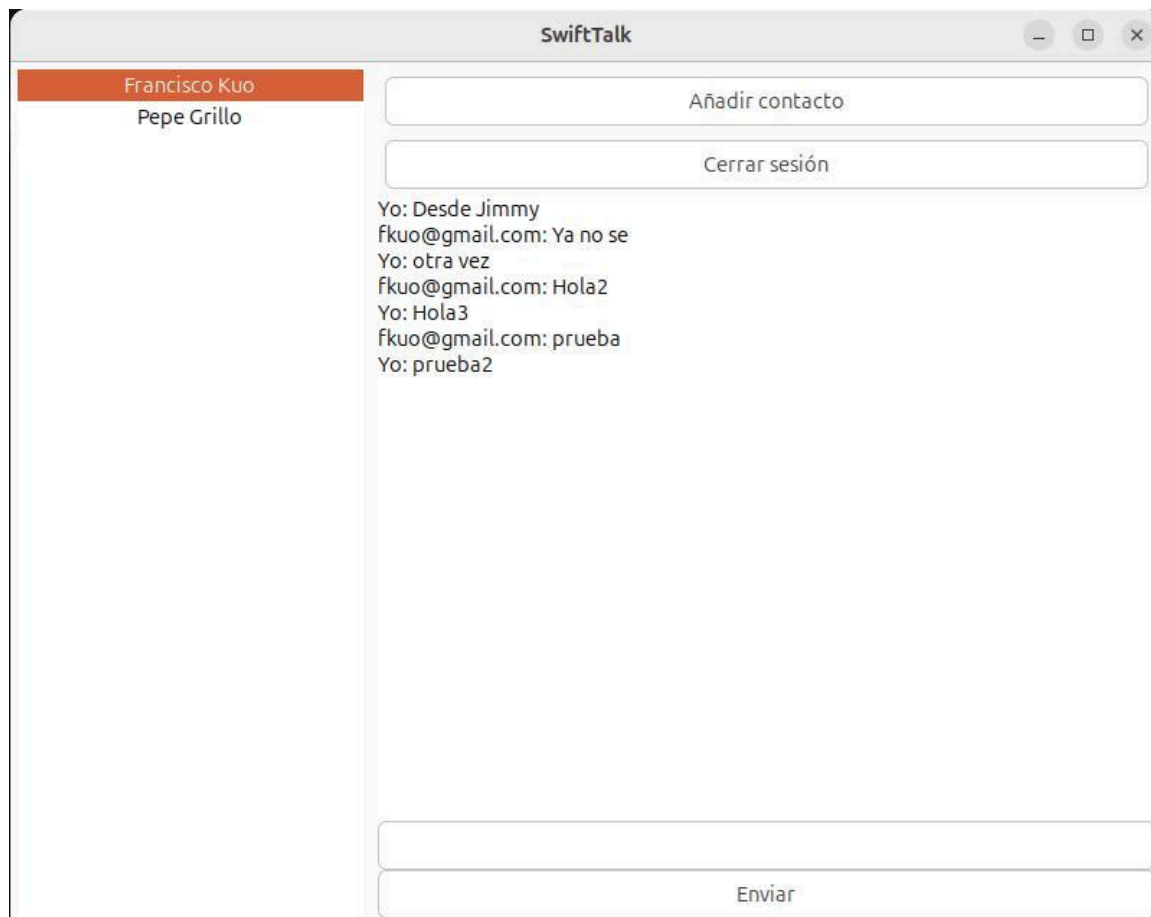
The screenshot shows a window titled "Inicio de Sesión" with standard window controls (minimize, maximize, close). It contains two input fields: "Correo:" with a text input box, and "Contraseña:" with a text input box containing the placeholder text "Ingrese su contraseña". Below these fields are two buttons: "Iniciar sesión" and "Registrarse".

Si se da click en el botón de registrarse, sale otra pantalla donde el usuario ingresa datos como su nombre, apellido, correo y contraseña. Al final al darle click a registrarse se agrega el usuario al sistema si los datos fueron ingresadas correctamente y este se devuelve a la pantalla de iniciar sesión donde puede ingresar sus credenciales para acceder al sistema.

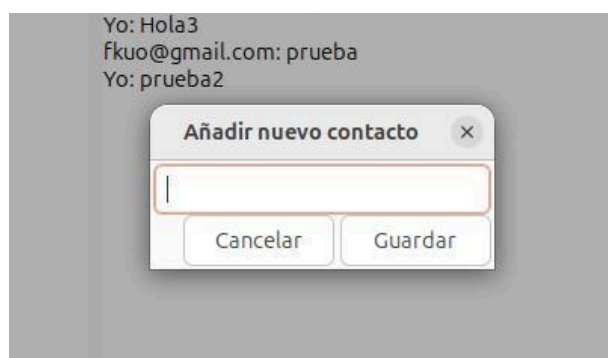


The screenshot shows a window titled "Registro de Usuario" with a close button. It contains five input fields: "Nombre:" with a text input box, "Apellido:" with a text input box containing the placeholder text "Ingrese su apellido", "Correo:" with a text input box containing the placeholder text "Ingrese su correo", "Contraseña:" with a text input box containing the placeholder text "Ingrese su contraseña", and "Confirmar Contraseña:" with a text input box containing the placeholder text "Confirme su contras...". Below these fields is a single button labeled "Registrarse".

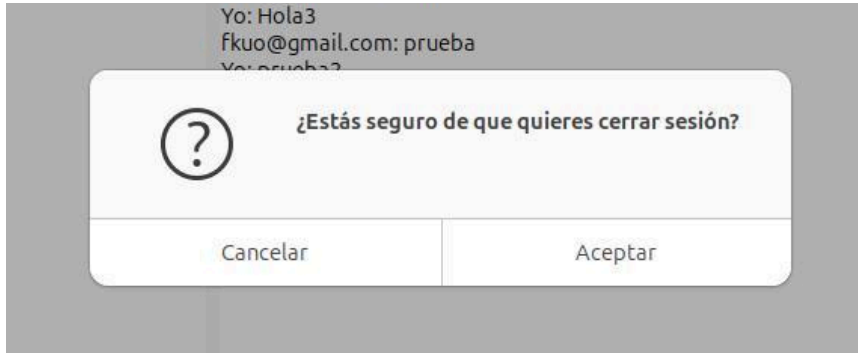
Esta es la pantalla principal donde se hacen los chats. En la parte izquierda se encuentran los contactos que tiene el usuario. Dándole click a estos contactos muestra los mensajes que se han enviado entre ellos. A la derecha están los botones de “Añadir contacto”, “Cerrar sesión”, la sección de mensajes, el campo para ingresar mensajes y el botón de “Enviar”.



Al darle click en “Añadir contacto” sale un popup donde se ingresa el correo del usuario al que se quiere agregar, luego se da en “Guardar” para confirmarlo.



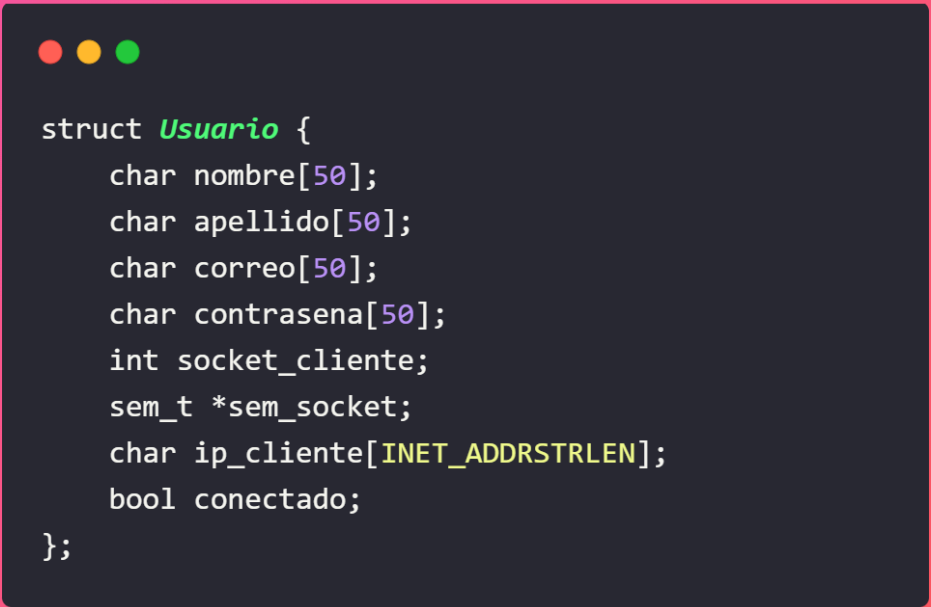
Si se da click en “Cerrar sesión”, sale esta notificación para que el usuario confirme si de verdad quiere salir del programa. Al darle aceptar el programa se termina y si no entonces se sigue ejecutando como normal.



Funcionalidad del Servidor

Estructuras y variables globales del Servidor:

<- struct Usuario ->

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code defines a C struct named 'Usuario' with the following fields: 'nombre' (char array of 50), 'apellido' (char array of 50), 'correo' (char array of 50), 'contrasena' (char array of 50), 'socket_cliente' (int), 'sem_socket' (sem_t pointer), 'ip_cliente' (char array of INET_ADDRSTRLEN), and 'conectado' (bool).

```
struct Usuario {  
    char nombre[50];  
    char apellido[50];  
    char correo[50];  
    char contrasena[50];  
    int socket_cliente;  
    sem_t *sem_socket;  
    char ip_cliente[INET_ADDRSTRLEN];  
    bool conectado;  
};
```

- Esta estructura permite mantener los datos esenciales de cada usuario dentro del servidor sin exponerlos a los clientes.

<- struct Mensaje ->


```

struct Mensaje {
    char correoEmisor[100];
    char correoReceptor[100];
    char contenido[512];

    // Constructor auxiliar (para uso fuera de la memoria compartida)
    Mensaje() {
        correoEmisor[0] = '\0';
        correoReceptor[0] = '\0';
        contenido[0] = '\0';
    }

    Mensaje(const string& emisor, const string& receptor, const string& msg) {
        strncpy(correoEmisor, emisor.c_str(), sizeof(correoEmisor));
        correoEmisor[sizeof(correoEmisor) - 1] = '\0';
        strncpy(correoReceptor, receptor.c_str(), sizeof(correoReceptor));
        correoReceptor[sizeof(correoReceptor) - 1] = '\0';
        strncpy(contenido, msg.c_str(), sizeof(contenido));
        contenido[sizeof(contenido) - 1] = '\0';
    }
};

```

- Esta estructura guarda cada mensaje con los correos del emisor y del receptor, y el contenido del mensaje.

<- struct SharedData ->

Es la estructura global que se guarda en memoria compartida para contener todos los usuarios y mensajes.

```

struct SharedData {
    Usuario lista_usuarios[100]; // Lista de hasta 100 usuarios
    Mensaje mensajes[100];      // Lista de hasta 100 mensajes
    int mensaje_count;           // Cantidad actual de mensajes registrados
    int user_count;             // Cantidad actual de usuarios registrados
};

```

- Esta estructura se coloca en un segmento de memoria compartida y es accesible por todos los procesos (hijos) del servidor.

- Se usa un semáforo global para evitar condiciones de carrera cuando varios procesos modifican datos a la vez.

>>Variables Globales del Servidor<<

```
SharedData *shared_data; // Apunta a la región de memoria compartida
sem_t *sem; // Semáforo global para sincronización
int server_fd; // Descriptor del socket del servidor
struct Usuario usuario_autenticado; // Usuario autenticado en el contexto actual del proceso hijo
```

- *shared_data*: contiene toda la información de usuarios y mensajes en memoria compartida.
- *sem*: sincroniza el acceso a la estructura compartida.
- *server_fd*: representa el socket principal del servidor (para aceptar conexiones).
- *usuario_autenticado*: variable local por proceso que guarda el usuario actualmente autenticado para esa conexión específica.

Lógica del servidor y conexiones(Uso de forks)

El servidor implementa un sistema de mensajería concurrente donde múltiples clientes pueden conectarse simultáneamente. Para lograr esto, se utiliza el mecanismo de bifurcación de procesos (`fork()`) para que cada conexión cliente sea manejada por un proceso hijo independiente, permitiendo comunicación simultánea sin bloquear el servidor principal.

Flujo Principal del Servidor

1. Preparación Inicial

Antes de que el servidor empiece a recibir clientes, necesita prepararse creando y organizando sus herramientas de trabajo:

- Memoria compartida: Es como una libreta que todos los procesos pueden usar para leer o escribir datos importantes. En este caso, ahí se guardan la

lista de usuarios y los mensajes que se intercambian. Ambas listas son estáticas debido a que para hacer memoria compartida, NO pueden ser dinámicas. En el caso de los mensajes donde se tiene

- Semáforo: Es un mecanismo que actúa como un semáforo real. Evita que dos procesos usen la libreta (memoria compartida) al mismo tiempo, para que no se confundan los datos.
- Carga de usuarios: Si ya hay usuarios registrados de antes, el servidor los carga desde un archivo al iniciar, así no se pierden los datos entre sesiones.

2. Creación del Socket del Servidor

Un socket es como el "puerto" por donde los clientes se van a conectar.

- El servidor crea un socket y lo configura para usar conexiones por internet (no locales).
- Lo enlaza a una dirección IP local y a un puerto específico (por ejemplo, el 8080 o el que diga el archivo de configuración).
- Luego lo pone en "modo escucha", esperando por nuevas conexiones.

3. Aceptando Clientes

Una vez que el servidor está listo:

- Se queda esperando conexiones nuevas. Cada vez que un cliente quiere conectarse, el servidor acepta esa conexión.
- Para no bloquearse mientras atiende a uno solo, el servidor crea un nuevo proceso (**fork**) para cada cliente o conexión:
 - El proceso hijo se encarga de hablar con ese cliente específico. Recibe sus mensajes, los procesa y le responde.
 - El proceso padre sigue esperando más clientes. No se encarga de hablar con ninguno directamente, solo los recibe y delega.
- Este mecanismo permite que varios clientes estén conectados al mismo tiempo, sin que unos interrumpan a otros.

Memoria Compartida

Se usa un sistema de memoria compartida para poder, tanto acceder a los mismos datos entre distintos clientes (o mejor dicho cada proceso subhijo que procesa a su único cliente). Se usa para la lista de usuarios y la lista de mensajes, este último utiliza un sistema de "pila" para tratarlo como una lista dinámica más que una lista estática. Usando, por supuesto, la lógica de índices de una lista estática.

El acceso a esta memoria está protegido por un semáforo, lo que garantiza que no haya conflictos o corrupciones de datos cuando varios procesos intentan acceder a la memoria al mismo tiempo. Esto optimiza la eficiencia en la gestión de usuarios y mensajes, permitiendo que el servidor maneje un número elevado de conexiones.

Lógica del Proceso Hijo (handle_client())

Una vez que el proceso hijo es creado por el servidor para atender a un cliente específico, comienza su trabajo dentro de esta función. Su única tarea es mantenerse en comunicación con ese cliente mientras dure la sesión.

1. Confirmar la conexión

Apenas inicia, el servidor le envía al cliente un mensaje para confirmarle que la conexión fue exitosa.

2. Esperar comandos del cliente

Luego, el servidor entra en un bucle while(true), donde espera que el cliente le envíe instrucciones. Cada instrucción viene en forma de un comando como "REGISTER", "LOGIN", "MSG", etc.

3. Procesar comandos recibidos

Una vez recibida una instrucción en formato de mensaje mediante un send() del cliente, se analiza para ver qué tipo de acción solicita el cliente. Aquí algunos ejemplos de lo que puede hacer el servidor:

- REGISTER: Registrar un nuevo usuario (crear su cuenta).
- LOGIN: Iniciar sesión si ya está registrado.
- MSG: Enviar un mensaje a otro usuario.
- CHECKMSG: Consultar si hay mensajes nuevos recibidos.
- GETUSER: Obtener información de otro usuario (como su IP).
- DISCONNECT: Desconectarse del sistema.

Cada uno de estos comandos llama a una función específica que hace el trabajo correspondiente.

Si el comando no es reconocido, se le envía un mensaje de error al cliente.

4. Manejo de errores o desconexión

Si en algún momento el cliente se desconecta o hay un error al recibir datos, el proceso termina su ciclo:

```
if (bytes_received <= 0) {  
    cerr << "Error al recibir el comando o cliente desconectado." << endl;  
    break;  
}
```

5. Cierre de la conexión

Al salir del bucle, se cierra el socket del cliente (porque ya no se va a usar más), y el proceso termina.

```
close(client_socket);
```

Lógica de registro y login de usuarios

- **Registro (register_user):**
- Comando: "REGISTER <nombre> <apellido> <correo> <contraseña>"
- Proceso:
 - Extrae los datos del comando.
 - Bloquea el semáforo y verifica si el correo ya está registrado.
 - Si no existe, crea un nuevo usuario, guarda la IP y lo agrega a la lista de usuarios.
 - Guarda la información de los usuarios en archivo.
 - Responde con un mensaje de éxito.
- **Login (login_user):**
- Comando: "LOGIN <correo> <contraseña>"
- Proceso:
 - Extrae los datos del comando.
 - Bloquea el semáforo y valida las credenciales.
 - Si las credenciales son correctas, marca al usuario como conectado y guarda la IP.
 - Responde con un mensaje de éxito o error.

Lógica de recepción y envío de mensajes

- **Envío de mensaje (procesarMensaje + addMSG):**
- Comando: "MSG <correo_destinatario> <contenido>"
- Proceso:
 - Valida el formato del comando.
 - Identifica el correo del emisor.
 - Verifica si el destinatario existe y está conectado.
 - Si está conectado, envía el mensaje; si no, lo guarda para enviarlo cuando se conecte.
- **Revisión de mensajes (revisarMensajes):**
- Proceso:
 - Bloquea el semáforo y revisa si hay mensajes para el usuario autenticado.

- Si hay mensajes, los envía a su cliente y elimina de la lista.
- Si no hay mensajes, responde con un mensaje de error.

Lógica de consulta de información de un usuario por correo

- Consulta (get_user_info):
- Comando: "INFO <correo>"
- Proceso:
 - Extrae el correo del comando.
 - Bloquea el semáforo y busca en la lista de usuarios.
 - Si encuentra al usuario, envía su información (nombre, apellido, correo).
 - Si no lo encuentra, responde con un mensaje de error.

Funcionalidad del Cliente

El cliente de este sistema de mensajería permite a los usuarios interactuar con otros mediante el envío y la recepción de mensajes, así como la gestión de sus contactos. A continuación se detallan las funciones implementadas.

Estructuras y Variables Globales del Cliente

<- struct Usuario ->

```
struct Usuario {  
    string nombre;  
    string apellido;  
    string correo;  
    string contrasena;  
};
```

Campos:

- nombre: El nombre del usuario.
- apellido: El apellido del usuario.
- correo: La dirección de correo electrónico del usuario, que también se utiliza para identificar al usuario de manera única.
- contraseña: La contraseña del usuario, utilizada para la autenticación.

Variable Global:

- usuario_autenticado: Una instancia de la estructura Usuario que almacena los datos del usuario actualmente autenticado.

<- struct Contacto ->

```
struct Contacto {
    string nombre;
    string apellido;
    string correo;
};
```

Campos:

- nombre: El nombre del contacto.
- apellido: El apellido del contacto.
- correo: El correo electrónico del contacto, que se utiliza como identificador para los mensajes y para cargar los chats asociados con el contacto.

Variable Global:

- lista_contactos: Un vector de Contacto que contiene todos los contactos almacenados por el usuario. Esta lista permite gestionar y acceder fácilmente a los contactos para enviar mensajes y guardar información sobre ellos.

<- struct MensajeChat ->


```

struct MensajeChat {
    string mensaje; // Contenido del mensaje
    int tipo;       // 0 para enviado por el usuario, 1 para recibido del contacto

    // Constructor para inicializar el mensaje y el tipo
    MensajeChat(const string& msg, int t) : mensaje(msg), tipo(t) {}
};

```

Campos:

- mensaje: Contenido textual del mensaje.
- tipo: Un valor entero que indica si el mensaje fue enviado por el usuario (0) o recibido de un contacto (1).

Constructor:

- El constructor inicializa un objeto *MensajeChat* con un mensaje y un tipo, permitiendo crear instancias fácilmente.

Variable Global:

- chatsPorContacto: Un mapa que asocia el correo de cada contacto con un vector de *MensajeChat*. Este mapa almacena todos los mensajes enviados y recibidos entre el usuario y sus contactos. La clave es el correo del contacto, y el valor es un vector que contiene los mensajes de chat correspondientes.

Enviar y Recibir Mensajes

- Envío de mensajes:
 - El cliente tiene la capacidad de enviar mensajes a otros usuarios mediante una comunicación con el servidor. Al enviar un mensaje, el cliente primero envía un comando al servidor con la dirección de correo del destinatario y el contenido del mensaje.

- Si el servidor confirma que el mensaje fue enviado correctamente, el cliente agrega el mensaje al historial de chat con ese destinatario. Si hay un error, se muestra un mensaje de error al usuario.
- Recepción de mensajes:
 - El cliente puede consultar al servidor para verificar si hay nuevos mensajes. Para ello, se envía un comando al servidor y se espera una respuesta. Si el servidor tiene mensajes nuevos para el cliente, estos se extraen de la respuesta y se muestran en pantalla. Los mensajes recibidos también se agregan al historial de chat correspondiente.
 - Los mensajes recibidos contienen información del remitente y del contenido, que luego se gestionan y se almacenan localmente en el historial de mensajes.

Cargar y Guardar Contactos

- Agregar Contactos:
 - Los usuarios pueden agregar nuevos contactos mediante la introducción del correo electrónico de otro usuario. El cliente envía un comando al servidor para obtener la información del usuario (nombre, apellido y correo) asociado al correo electrónico proporcionado.
 - Si el contacto es válido y no es el propio usuario, el cliente lo agrega a la lista de contactos. En caso de que el contacto sea el propio usuario, se actualizan sus datos locales.
- Guardar Contactos:
 - Una vez que el cliente tiene una lista de contactos, estos se pueden guardar en un archivo local específico para el usuario. Este archivo contiene los datos de cada contacto en formato CSV (nombre, apellido, correo) y se almacena con el nombre del usuario autenticado.
 - Esto asegura que los contactos puedan ser persistentes incluso después de cerrar la aplicación.
- Cargar Contactos:
 - El cliente puede cargar los contactos previamente guardados desde el archivo local. Al abrir la aplicación, el cliente verifica si existe un

archivo con los contactos guardados y los carga en la memoria del cliente.

- Si no se encuentra ningún archivo o si el usuario aún no tiene contactos guardados, se informa al usuario que no hay contactos disponibles.

Carga y Guardado de Chats

Guardar Chats

El objetivo de la función de guardar chats es almacenar todos los mensajes que el usuario ha enviado y recibido en un archivo para que puedan ser recuperados más tarde.

Estructura de Datos Involucrada:

- Los mensajes se almacenan en una estructura llamada `chatsPorContacto`, que organiza los mensajes por contacto. Es un mapa donde cada contacto (identificado por su correo) tiene asociado un historial de mensajes. Este historial es una lista de mensajes, donde cada mensaje tiene un texto y un tipo (si fue enviado o recibido).

Proceso de Guardado:

- Primero, se genera un nombre de archivo único para el usuario, que utiliza su correo electrónico para crear el nombre del archivo. Esto asegura que cada usuario tenga su propio archivo para guardar los chats.
- El archivo se abre en modo de escritura. Si el archivo no se puede abrir, el sistema muestra un error.
- Luego, se recorre el mapa `chatsPorContacto`, que tiene los correos de los contactos como claves y los mensajes correspondientes como valores. Para cada contacto, el sistema escribe el nombre del contacto en el archivo seguido de los mensajes que el usuario ha intercambiado con ese contacto.

- Los mensajes se guardan con el tipo de mensaje (si fue enviado o recibido) y el texto del mensaje, separados por un delimitador (por ejemplo, un punto y coma).
- Al finalizar el historial de cada contacto, el sistema agrega un marcador indicando que ese chat ha terminado.
- Finalmente, se cierra el archivo, y los chats del usuario han sido guardados con éxito.

Cargar Chats

La función de cargar chats permite recuperar los mensajes que el usuario ha guardado previamente en un archivo y restaurarlos en la estructura de datos adecuada para que el usuario pueda ver su historial de conversaciones.

Estructura de Datos Involucrada:

- Similar al proceso de guardado, los mensajes se almacenan en un mapa (chatsPorContacto), donde cada contacto tiene su propio historial de mensajes.

Proceso de Carga:

- Primero, el sistema genera el nombre del archivo donde se encuentran los chats del usuario. Este archivo tiene el mismo formato que el archivo usado para guardar los mensajes, y su nombre está basado en el correo del usuario.
- El archivo se abre en modo de lectura. Si no se puede abrir, se muestra un mensaje de error.
- El sistema lee el archivo línea por línea. Cuando encuentra una línea que indica el comienzo de un chat con un contacto (por ejemplo, "Contacto: [correo del contacto]"), guarda esta información como el nuevo contacto y comienza a almacenar los mensajes correspondientes.
- Cada mensaje se lee, se separa en dos partes: el tipo (si fue enviado o recibido) y el contenido del mensaje. Luego, estos se agregan a la lista de mensajes de ese contacto.

- Cuando se encuentra una línea que indica el fin del chat con un contacto (por ejemplo, "---- Fin de chat con [contacto] ----"), se guarda el historial de mensajes para ese contacto en el mapa chatsPorContacto.
- Al final del archivo, si el último contacto tiene mensajes, también se guarda el historial de mensajes en el mapa.

Resultado:

- Al finalizar, los mensajes de todos los contactos se han cargado correctamente en el mapa chatsPorContacto, y el usuario puede acceder a ellos como si nunca se hubiera cerrado la aplicación.

Interacción con el Cliente

El cliente puede interactuar con estas estructuras y variables de la siguiente manera:

- Agregar contactos: Cuando el usuario agrega un contacto, la información del contacto (nombre, apellido, correo) se guarda en lista_contactos.
- Enviar mensajes: Los mensajes enviados por el usuario se almacenan en chatsPorContacto bajo el correo del contacto correspondiente. La función agregarMensajeAlChat se encarga de añadir un nuevo mensaje a la lista de mensajes del contacto.
- Recibir mensajes: Los mensajes recibidos de otros contactos se agregan de manera similar en chatsPorContacto, pero con el tipo de mensaje 1 (indicado como recibido).
- Guardar y cargar mensajes: Los chats del usuario se pueden guardar en un archivo utilizando la función guardarChatsEnArchivo, y se pueden cargar nuevamente utilizando la función cargarChatsDesdeArchivo. Estos chats se organizan por contacto y se almacenan en el mapa chatsPorContacto.

Comunicación entre Cliente-Servidor

1. Establecimiento de Conexión (Cliente - Servidor)

- Cliente: El cliente inicia la conexión con el servidor enviando una solicitud de conexión a través de un socket.
- Servidor: El servidor está escuchando en un puerto específico esperando a que los clientes se conecten. Cuando un cliente intenta conectarse, el servidor acepta la solicitud de conexión y establece un canal de comunicación.

El socket de conexión se utiliza para enviar y recibir datos entre el cliente y el servidor.

2. Autenticación del Usuario

- Cliente: Una vez establecida la conexión, el cliente se autentica con el servidor enviando sus credenciales, como el correo electrónico y la contraseña.
- Servidor: El servidor valida las credenciales del cliente. Si la autenticación es exitosa, el servidor responde con un mensaje de éxito; en caso contrario, envía un mensaje de error.

3. Interacción de Mensajes

La comunicación de mensajes es la parte central del proceso:

- Envío de mensajes:
 - Cliente: El cliente crea un mensaje con el contenido y la dirección del destinatario. Luego, envía el mensaje al servidor utilizando un formato específico.
 - Servidor: El servidor recibe el mensaje y verifica si el destinatario está disponible. Si el destinatario está conectado, el servidor transmite el mensaje al cliente correspondiente. Si el destinatario no está conectado, el servidor puede almacenarlo para enviarlo cuando el destinatario se conecte.

- Cliente: El cliente que envió el mensaje recibe una confirmación del servidor de que el mensaje fue entregado correctamente.

Recepción de mensajes:

- Cliente: El cliente puede consultar si tiene nuevos mensajes. Esto se realiza mediante una solicitud al servidor, donde el cliente pide al servidor que le envíe los mensajes nuevos que ha recibido.
- Servidor: El servidor verifica si hay mensajes nuevos para el cliente. Si existen, el servidor envía los mensajes de vuelta al cliente en un formato adecuado.

4. Verificación de Mensajes Nuevos

- Cliente: El cliente puede enviar una solicitud al servidor para comprobar si hay nuevos mensajes. Esto se hace mediante un comando específico como "CHECKMSG".
- Servidor: El servidor verifica si el cliente tiene nuevos mensajes en su bandeja de entrada. Si hay mensajes, el servidor los envía al cliente, junto con los detalles del remitente.

5. Gestión de Contactos

- Agregar un contacto:
 - Cliente: El cliente puede agregar un nuevo contacto a su lista enviando la dirección de correo del contacto al servidor. El servidor verifica si el usuario existe.
 - Servidor: El servidor responde si el usuario existe o no. Si el usuario existe, se agrega el contacto a la lista del cliente.
- Guardar y cargar contactos:
 - El cliente puede guardar la lista de contactos localmente en un archivo para cargarla en futuras sesiones.
 - El cliente también puede cargar los contactos desde un archivo si ya están guardados previamente.

6. Enlace entre Mensajes y Contactos

- Cliente: Cada mensaje enviado o recibido se asocia a un contacto específico, y los mensajes se almacenan localmente en una estructura de datos como un mapa (map<string, vector<MensajeChat>>), donde cada contacto tiene un historial de mensajes.
- Servidor: El servidor maneja las comunicaciones, pero no guarda permanentemente los mensajes. Solo procesa y retransmite los mensajes.

7. Desconexión

- Cliente: Cuando el cliente desea desconectarse, envía una señal de cierre de conexión al servidor.
- Servidor: El servidor recibe la señal de desconexión, termina la sesión y cierra el canal de comunicación para ese cliente. Y finaliza de manera adecuada el proceso hijo (Él mismo)

8. Manejo de Errores

En cualquier parte de la comunicación, pueden ocurrir errores, como problemas de conexión, credenciales incorrectas, o fallos en la transmisión de datos. En tales casos, el servidor y el cliente intercambian mensajes de error para notificar el problema. El cliente siempre maneja esos casos en el que el mensaje sea un mensaje de error y los maneja según sea el caso.